第四部分 · 记忆的表示与存储架构

分层记忆与混合架构

为什么智能体不能只用一种记忆存储?

Hierarchical Memory · Hybrid Retrieval · Memory Management

> 长期记忆不是“越多越好”，
> 而是让正确的信息，在正确时刻，
> 以可核验的方式支持正确行动。

零基础理论课 · 16张机制原理图 · 12道思考题与详细答案

v1.0 | 2026-10-18 | 纯理论，无实验

第20讲｜分层记忆与混合架构

《智能体记忆系统：从零基础到科研理解》 · 第20/32讲 · 第四部分：记忆的表示、存储架构与检索机制

副标题：为什么智能体不能把所有经历都塞进一种存储？

本讲的核心问题： 如果文本能保存原文、向量能找到相似语义、图能表达关系、数据库还能保存状态，那么长期科研助手到底应该选哪一种？是四种全部部署，还是只选一种？一件新事发生后，哪些信息必须立即改变，哪些可以过一段时间再整理？多个检索结果说法不一致时，又怎样得到正确的当前判断？

阅读约定： 这是纯理论课程，不安排编程、安装、运行实验或收费调用。本文所有“D01—D30”的事件、Token、毫秒和成本数字均为明确标注的**教学构造**，用来讲清逻辑与计算，不是我们已运行的实验，也不代表真实授权。文中引用的研究成果与本文提出的教学抽象模型将明确区分。

学习目标与全局地图

学完应当能够：①区分记忆功能分类、逻辑存储层、物理数据库三条维度；②画出一个可追溯的分层长期记忆架构；③解释当前上下文、线程检查点、权威事件/状态、检索索引和历史证据分别负责什么；④推演新决定从写入到更新不同检索视图的完整过程；⑤理解查询路由和混合检索；⑥分析多索引不一致造成的错误；⑦用准确性、延迟、成本和维护复杂度判断是否应增加一层存储；⑧从错误行为反查系统中的故障位置。



图1 分层与混合系统的总知识地图｜原创教学机制示意

> **本讲的总原则：** 分层不是“数据库越多越高级”，而是根据信息**使用频率、更新时效、查询方式、可靠性和成本**分配不同责任；混合不是让多个结果简单投票，而是在同一个事实来源与有效状态约束下，使用互补的检索方式。

一、从一个困境开始：同一个科研智能体为何需要不同“记忆抽屉”？

继续上一讲的虚构科研助手。它在30天内接收到：

- **D01**：研究者确定当前工作首先开展**零费用离线调研**。
- **D03**：AI提出未来可购买API额度扩大实验。它只是**建议**，没有获批。
- **D08**：研究者明确“暂不批准新增付费实验”。这是**有效决定**。
- **D11**：自动化工具出现一次HTTP 504，原始日志被保存。

- **D18**：查看网关与重试记录后，形成“这次504可能涉及网关超时”的**有证据、有限范围的诊断**。
- **D20**：研究者再次明确“未经新许可不得恢复新增付费API调用”。
- **D30**：研究者问“下一步能否重启那些收费实验？如果不能，依据是什么？此前的504故障说明了什么？”

把这30天所有对话塞回模型上下文：资料庞大、每轮昂贵，而且旧的“AI建议”可能比正式决定更容易被模型复述。只保存一页滚动摘要：虽然便宜，却可能压掉“未批准”的否定词、不同时间与原始证据。只使用向量库：容易召回语义接近的D03建议，却不能仅凭向量距离识别它已经不具有决策效力。只用关系图：能找到“决定限制实验”的边，但仍需要原始D20消息证明那条边没有被误抽取。

所以需要的不是四份互相争当真相的数据库，而是一个有“权威原始依据”、多个面向不同问题的视图和明确控制规则的系统。



图2　同一个科研项目中的不同检索诉求｜原创教学机制示意

1.1 三种根本不同的问题

- “**当前能不能收费？**”要求快速定位**当前有效决定**，且应由外部控制器核实授权。高相似文本不是许可。
- “**D11发生了什么？**”要求时间明确的**原始事件与日志**，不能只听AI的总结。
- “**这些故障与哪些决策有关？**”涉及**关系与多跳查询**，图或结构化关联可能比单段文本更方便。

这里已出现分层的第一个理由：**不同问题的正确访问路径不相同**。

二、先把三个维度分开：类型、层次和物理实现

第10讲讲工作、情景、语义、程序性记忆，主要是在问“内容发挥什么认知功能”；第17至19讲讲文本、结构化、向量、图，主要在问“内容如何表示和访问”；本讲的“层次”还要进一步问“哪些信息应该立即放进当前工作区，哪些可以跨会话保存，哪些只在需要时取回来”。

维度	它回答的问题	举例	不应该推导出的结论
功能类型	智能体用它做什么?	情景事件、语义事实、程序性流程	四类功能必须用四种数据库
逻辑层次	何时装载、多久保留?	当前上下文、线程检查点、长期记忆、历史归档	快的层只能存“工作记忆”
表示与索引	怎样存、怎样找?	原文、键值、全文索引、向量、图	每种表示必须独占一个服务器

维度	它回答的问题	举例	不应该推导出的结论
物理实现	数据实际放在哪里?	文件、关系库、向量索引、对象存储	买了图数据库就自动拥有图记忆能力

一个关系数据库可以保存原始消息、结构化当前状态、向量列和关联边；一个独立向量索引可以给同一条记录提供语义访问入口。因此，**“四种视图”不等于“四台服务器”**。



图3　四个互相正交的系统设计维度｜原创教学机制示意

三、分层的最直观类比：书桌、待办本、档案柜与目录卡

把一个长期科研助手想象成每天处理大量资料的研究人员。书桌上只能放当前要读的几页；待办本记录进行中的任务；档案柜保存正式会议纪要与全部实验日志；目录卡帮助快速定位档案位置。即使目录卡写着“允许调用”，研究人员仍要翻原始批准文件确认内容和范围。



图4　当前工作区、检查点、权威记录和派生检索｜原创教学机制示意

3.1 第一层：当前上下文（Active Context）

模型每次推理真正“看得到”的输入包括：当前用户问题、系统/控制约束、当前任务信息、检索进来的若干记忆片段、工具结果等。它容量有限，而且这次输入使用结束之后，**不能自动假定所有内容被写入了长期记忆**。上下文不是存储库，是当前工作的**可见信息集合**。

典型内容：D30提问、D20有效限制的简要表示、D11/D18已核验片段、当前任务步骤。它不应该无差别装载30天全文。

3.2 第二层：任务或会话状态（Thread State / Checkpoint）

某项任务执行到第4步，已读取哪些文件？当前子任务是什么？哪些工具结果已经返回？这些用于**恢复执行**的信息，通常由线程状态和检查点保存。它与长期“研究者偏好”不是同一职责。

例如：正在阅读6篇论文，已完成4篇，剩余2篇；哪一个文件已校验哈希。检查点可以让被中断的工作恢复，但它**不意味着事实被核实，也不应当代替跨任务长期知识**。LangGraph在官方文档中明确区分线程级checkpointer与跨线程store。[4]

3.3 第三层：权威事件与当前有效状态（Canonical Record）

这层保存“原始经历”及其经过明确更新后的有效状态：D03是AI建议、D08/D20是研究者决定、D11是工具原始记录、D18是诊断结论与证据链。最好让**原始事件**与**当前状态视图**同时存在：当前状态回答“现在有效什么”，事件历史回答“为什么、何时改变”。

“权威”指这份存储是系统**统一读取状态的规范记录位置（source of truth）**；它不意味着存进去的每个事实都已经客观证明。未经核验的断言仍要标注为候选或待核验。

3.4 第四层：面向不同问题的派生索引（Search Views）

搜索时不一定能直接扫描全部事件；可以维护全文倒排索引（精确词项）、向量索引（语义相关候选）、结构化字段索引（项目、时间、状态）及图关系索引（跨事件路径）。它们是**帮助找到权威记录的视图**，不是能互相覆盖的四个真理来源。

3.5 第五层：历史资料、归档及备份（Evidence / Archive / Backup）

旧版本、原文PDF、完整日志、附件、存储快照是重要的审计材料。**备份服务于灾难恢复，不能等同在线记忆；档案有证据价值，也不能默认每轮都发送给LLM。**



图5　五种信息责任与逻辑层次｜原创教学机制示意

四、为什么MemGPT要借鉴操作系统的虚拟内存?

操作系统把经常使用的数据放到快速且有限的内存，把大量数据保存在容量更大的磁盘。应用程序似乎在访问一片连续空间，但操作系统实际在后台管理数据的装入、换出和页面映射。MemGPT论文借用了**虚拟上下文管理（virtual context management）**思想，设计多层记忆及内存操作，让智能体能够在有限模型上下文内处理更长的历史。[1]



图6　操作系统分页思想与有限上下文管理｜原创教学机制示意

4.1 “映射关系”到底是什么?

操作系统概念	面向智能体的有用类比	必须注意的差异
内存RAM	模型当前可访问上下文	LLM上下文不是可任意按地址读写的RAM
磁盘文件	外部持久化历史与事实	外部库里的内容不会自动进入上下文
页调入	从存储检索必要片段并加入提示	检索可能找错、摘要可能变形
页换出	移出当前上下文，必要内容归档	移走不等于已经成功写入或仍可重建
页面替换策略	选择哪些片段继续保留	重要性依任务语义，不能仅按最近使用时间

最容易误解的点： 操作系统读取某块磁盘数据后通常能获得原字节，智能体从语义记忆检索出一段摘要却未必忠实于原文。**虚拟内存是启发性比喻，不是二者计算保证的等价证明。**

4.2 主动管理和被动装载

一种设计是：控制器在每轮任务前自动选择要检索哪些记忆；另一种设计是：智能体可以调用受控的记忆搜索、更新等工具主动管理自己的外部状态。MemGPT更接近后者的典型研究路线，但实际系统常结合两种。无论哪种，**模型提出的写入/删除/调用都必须经过控制层权限与一致性检查。**

五、混合架构的工作方式：统一记录，多条检索路径

想象D30问：“我们现在是否可以恢复某项收费实验？以前暂停的原因是什么？”这个问题至少同时包含**当前状态查询**与**历史来源解释**两项子任务。如果只用同一个向量相似度排序，很容易把两个问题混成一锅粥。

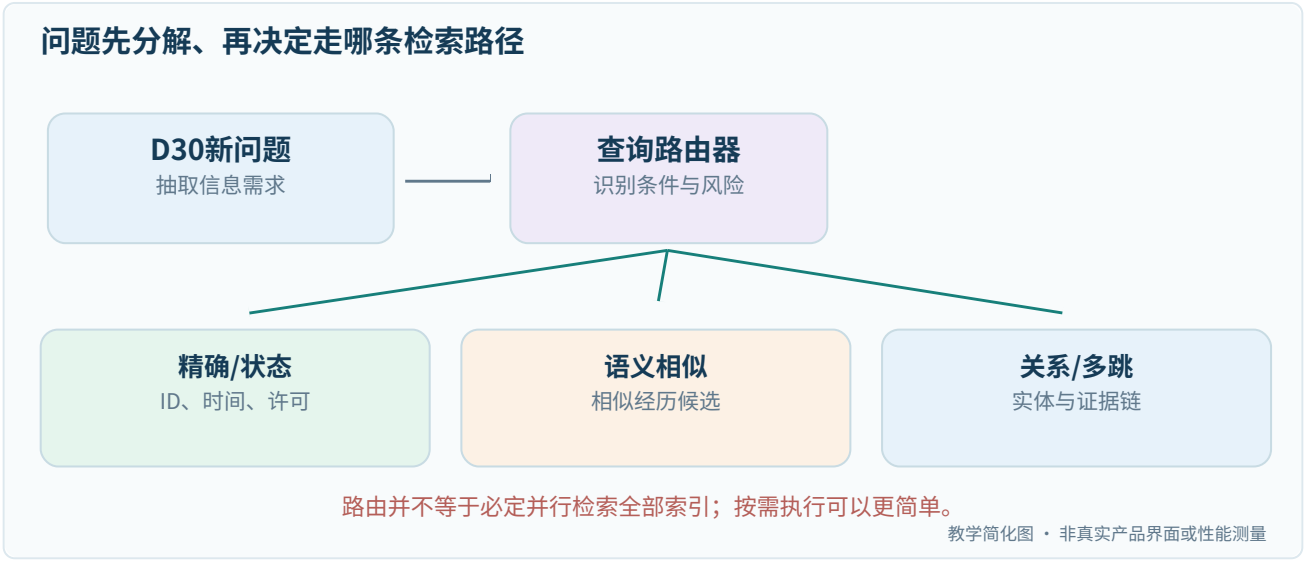


图7　查询路由及按需混合检索｜原创教学机制示意

5.1 查询路由（Query Routing）是什么?

查询路由就是先判断“这道题需要哪些证据”，再选合适访问路径：

- 查准确标识符、API状态码、某条原话：先用**关键词/全文索引**或精确ID。
- 查“现在允许什么”“哪个版本生效”：先用**结构化状态查询+时间和作用域过滤**。
- 查语义类似经历：用**向量索引**召回候选，再回源核查。
- 查多次决定、人物、证据之间的路径：用**关系/图遍历**，但仅在确有多跳需要时启动。

路由器本身也可能判断错，因此一个稳健设计应允许**简单查询优先**、必要时补充第二路检索，而不是把所有查询都强制送进最贵的混合系统。

5.2 “各路索引融合”不等于可信度投票

混合检索（Hybrid Retrieval）可以联合关键词排序与语义排序；可按分数归一化，也可使用RRF等只依赖名次的融合方法。第08讲已学过RRF，这里只复习其直观含义：一条候选若在多条检索路径中都排得靠前，就有理由被优先看见；**但排得靠前不能证明它现在有效，也不能证明它是真实的**。

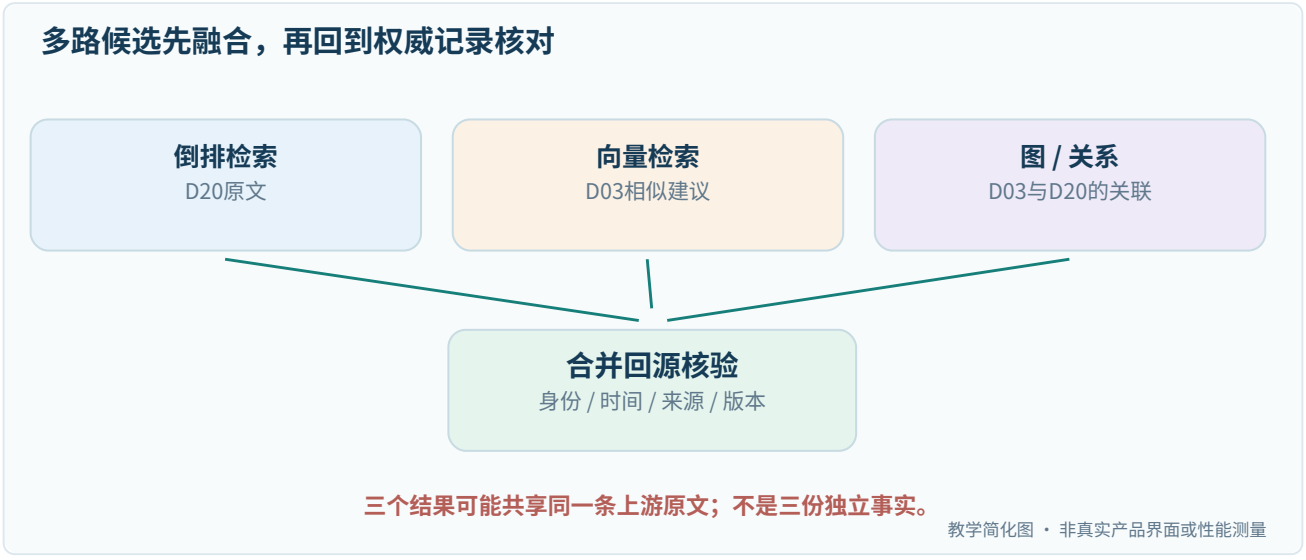


图8　不同检索视图合流与来源去重｜原创教学机制示意

5.3 候选的正确下一步：回源、去重、状态核验

假设向量搜索找到D03“建议扩大付费实验”，全文索引找到D20“未批准恢复”，图检索找到D03与D20有关联。系统不应该简单判断“支持恢复1票、反对恢复1票”，而应先检查：谁说的？当时是什么角色？哪条消息是建议，哪条是实际决定？D20覆盖的时间和项目是什么？是否存在后续有效授权？只有通过这些检验，才能向模型装载经过整理的证据。

5.4 为什么索引必须引用同一个 memory_id?

一条事件E20有原文、结构化决策、语义向量、关键词倒排入口和可能的图边。不同表示都应该带回稳定的`memory_id`/`event_id`及版本，避免将不同来源的相似文本当作同一事实。**“多视图共享身份”能帮助一致性，但不能自动解决写入顺序、跨库延迟和错误提取。**

六、更新才是架构难点：一条新决定要改哪些地方?

读的时候分层容易理解，写的时候更容易出问题。假设D26研究者在某项目里批准了“仅限10次、≤20元的一次性验证”（纯教学情景）。需要更新的是相应项目的权限状态与新事件；旧的全局“不自动付费”规则可能仍然有效，因为两者范围不同。

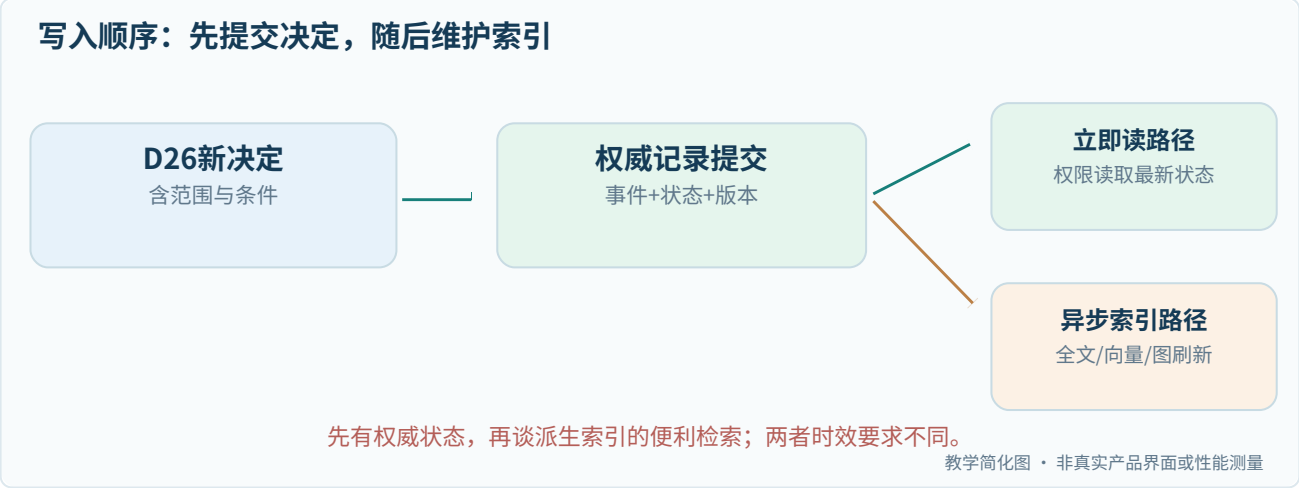


图9　关键状态提交与派生索引的差异化时效｜原创教学机制示意

6.1 关键状态需要先确认后生效

一种稳妥的教学架构是：先把D26原文及其作用域写入**规范事件库**，通过授权验证后，在同一权威事务范围内更新**当前状态**，记录版本号；只有事务成功，才宣布该状态生效。若某项高风险操作的权限需要读取最新状态，**不可单纯依赖可能滞后的向量索引、图缓存或LLM口述。**

6.2 派生索引何时更新?

权威状态成功提交后，再把“事件发生了变化”发送给派生索引构建者，更新全文索引、向量和图。因为多种引擎通常不共享一个跨库事务，它们可能在短时间内出现版本差异。**系统必须明确哪些查询允许读到稍旧的索引，哪些判断必须回权威库核验。**

6.3 常见做法：事件日志/Outbox/版本检查（教学概念）

为了减少“双写一半失败”的风险，可以在同一数据库事务中提交业务状态和一条待处理事件（outbox），由后台工作者可靠消费，更新派生索引；配合幂等ID、失败重试和死信/补偿机制。它是**常见系统工程模式**，不是所有记忆论文都采用的唯一标准，也不保证绝对零延迟。注意：需要定义严格的操作授权，不能让异步索引更新成为绕过权限检查的借口。



图10　分离权威提交和可靠的异步索引维护｜原创教学机制示意

6.4 “更新记录成功”和“下游全部更新成功”是两种状态

若规范库版本为`v12`，但向量索引还停留在`v10`，就可能先召回旧语义片段。系统可采用**版本戳**、有效状态回源过滤、读后验证、增量修复或短期阻断；不能因为查询返回了Top-K结果就假设那些片段适用于当前任务。

6.5 删除比新增更复杂

如果用户要求移除某条记忆，系统不仅需要处理当前主数据，还要处理向量索引、关键词索引、图关系、缓存、副本、备份留存政策与合规要求。**让模型不再看到某条信息，与系统真正删除全部相关副本，是两个不同的目标。**

七、完整动态例子：从D30提问到可靠回复和下一轮记忆更新

现在把所有模块连接起来，以D30询问“能否恢复付费实验，为什么过去暂停”为例。以下流程为**教学架构**，不代表任何一个具体产品的精确实现。

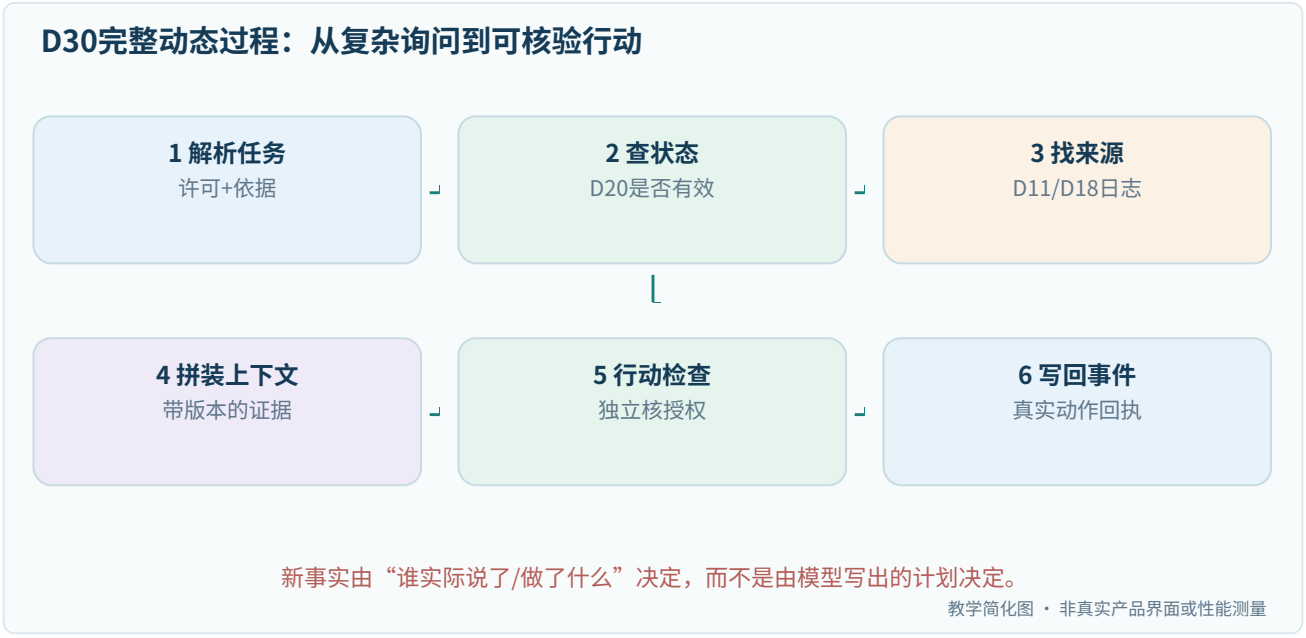


图11　六阶段科研助手任务与记忆循环｜原创教学机制示意

阶段1：识别任务。 模型或路由器识别出两个信息需求：①当前付费许可状态；②历史暂停和故障的证据。

阶段2：查询权威状态。 根据项目ID、当前时间和作用域，查到D20禁令仍有效；如果D26存在，只评估它明确涵盖的有限范围。

阶段3：召回历史。 关键词找到D11的504日志，向量找到D18的诊断说明，关系索引连到相关事件；回源核对出“这次504可能与网关超时有关”，而不是“整个实验失败的科学原因必然是网络”。

阶段4：组装工作区。 将D30问题、有效状态、D11与D18证据片段、来源和不确定性装进有限上下文。旧D03 AI扩样提议可作为历史背景，但明确标为“未获批准的建议”。

阶段5：生成与行动。 模型可以提出“先离线检查，若要付费需要新的明确批准”；控制器在实际执行任何费用操作前再次核验当前授权。即使模型误写成“可以开始”，也不能绕过执行边界。

阶段6：记录结果。 把本轮用户最新决定、工具实际回执与任务状态分别写回；AI自己的建议仍是建议，不能被系统自动升级成批准。

7.1 两个关键不变量（Invariant）

不变量是我们希望系统无论经过多少轮、多少次检索都维持的性质。

- **不变量A：任何检索视图都不能单独改变权威授权状态。** 它可以提供候选证据，但不是授权写入接口。
- **不变量B：任何重要结论都应能回到可识别的来源与版本。** 若来源不存在或版本不一致，系统应限制置信程度或明确标记未知。

上述不变量属于**架构设计规范**，不是对当前所有产品已经满足该性质的经验断言。

八、为什么复杂系统不一定更好？简单基线必须保留

假设研究任务只是让智能体记住3条用户偏好、1条当前项目约束和10份可全文搜索的笔记。一个带稳定ID、来源、时间字段的结构化表，加简单关键词检索，也许已能解决绝大多数问题。强行加入向量服务、图数据库和定期LLM总结，可能带来新的版本差异与额外费用。

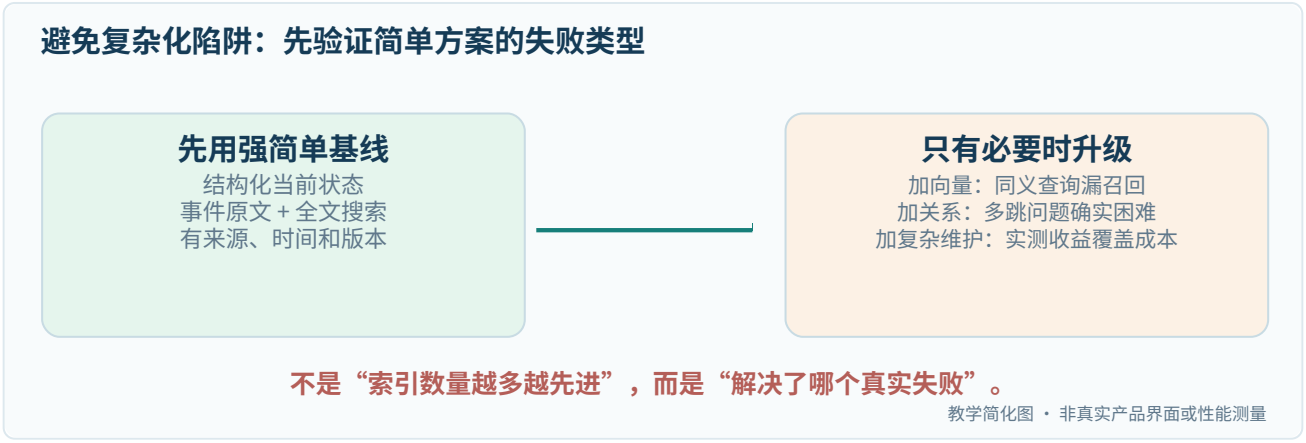


图12　混合架构的渐进式选择标准｜原创教学机制示意

8.1 按“需求→结构”做选择，而不是按“流行框架→需求”

真实问题	优先考虑的简单方案	什么时候可能值得升级
读取用户当前约束	结构化当前状态 + 原始证据	需要多条件跨来源推理时
找API报错原文	全文索引/精确字段过滤	同义描述很多、关键词不稳定时加向量
找相似历史故障	向量 + 回源 + 状态过滤	需要跨事件因果/关系解释时加图
找项目与多篇论文的关联	关系表或轻量图结构	高频复杂多跳遍历时升级图引擎
继续未完成任务	检查点 + 任务状态	跨任务复用经验时增加长期Store

8.2 架构扩展通常会增加哪些负担?

新增一层索引不仅多了一份数据：还需要数据模型、索引构建、写入策略、版本同步、延迟处理、备份、故障恢复、观察指标和迁移方案。复杂度不是只看“有多少服务器”，还要看**多少种状态可能彼此不一致**。

8.3 学习层面的增量原则

一个便于科研思考的方式：先问“单一强简单基线的真实失效是什么”，再问“新增某一层是否针对该失效给出明确的可检验机制”。例如：若主要问题是“旧决定覆盖新决定”，仅增加图数据库不一定解决，**先补上时间、来源、状态核验机制**可能更关键。系统结构多，并不自动意味着科研方法更新颖或性能更强。

九、资源与成本：一个能够独立复算的例子

9.1 当前上下文预算

假设模型允许当前轮输入+输出合计`12,000 tokens`，课堂计划保留：当前规则和任务约束`800`、当前有效状态`500`、4段历史事件每段`180`、两段原始证据每段`250`、本轮用户问题`250`、预留回答`1,200`。需要的总预算为：

`800 + 500 + (4×180) + (2×250) + 250 + 1,200 = 3,970 tokens`。

这里真正装入的输入部分是`2,770 tokens`，预留输出`1,200 tokens`，合计`3,970`。**这是一组教学预算，不是特定平台保证的Token分配；实际系统提示、工具消息和推理开销还需另外记账。**

若全历史本轮要放入`16,000 tokens`，会超出课堂上假定的`12,000`限制；分层检索至少为这道题提供了一个可以控制预算的结构，但它可能遗漏关键线索，不保证任何任务都优于全文回放。



图13　当前工作区的教学预算｜原创教学机制示意

9.2 端到端延迟为什么要分层计算?

假设一次请求的教学延迟是：查询向量编码`40 ms`、索引召回`70 ms`、回源与状态核验`35 ms`、LLM生成`600 ms`。总计：`40 + 70 + 35 + 600 = 745 ms`。若复杂关系检索增加`120 ms`，总计`865 ms`。某些系统可并行执行查询，但此处为了易懂使用**串行求和模型**。增加图模块后，若正确率没有实质提升，额外的120ms就可能是不值得的工程负担。

9.3 生命周期成本比单次检索更重要

系统每天做100次查询；若新增索引每次多消耗5毫秒，但每天还需后台做索引更新、重新嵌入、构图、存储与维护，判断其成本时应一起考虑：

`总成本 = 构建与迁移 + 日常写入维护 + 查询与生成 + 失败修复 + 监控与恢复`。

但这些成本单位可以是费用、延迟、人工、存储、失败率，需要**分别报告或经过明确权重归一化**，不能直接把人民币、毫秒和准确率当成同一种数字相加。

十、可靠性评估：不要只看Top-K检索命中率



图14　系统级可靠性与索引级质量之间的区别｜原创教学机制示意

层级	需要回答的问题	可能的指标/审查
权威事件层	原始证据完整吗？身份和时间对吗？	来源覆盖、字段完整、版本一致
写入维护层	应记录的信息保留了吗？错误主张被核验了吗？	关键记忆召回、错误写入率、纠错传播
检索索引层	需要的候选是否被找回？	Recall@K、候选新鲜度、检索延迟
上下文构造层	正确证据是否送到模型眼前？	证据覆盖、Token开销、冲突标记
推理与行动层	结论是否可信、行为是否在授权内？	事实准确、任务成功、越权动作数
资源运行层	系统是否稳定、成本是否合理？	p50/p95延迟、调用成本、更新积压

10.1 为什么“图检索成功”仍可能回答错？

可能路径上某条边来自错误提取；可能旧边已经失效；可能正确证据被截断；也可能模型读懂后仍错误推理。这四种错误分别发生在**建图、状态更新、上下文构造和模型利用**阶段，不能统统称作“图检索失败”。

10.2 为什么“多Agent都同意”也不够？

三个智能体若引用同一个错误摘要，它们的共识并不提供三份独立证据。混合架构的价值之一是能**把不同表示对应回同一个原始记录**，从而看出所谓多个来源其实可能来自同一条上游信息。

十一、代表性研究及其不同的架构思想

这里讲的是设计思想，**不是将各论文结果做未经条件对齐的排名**。

代表性工作	从中学习什么	不应误读为
MemGPT（2023）[1]	虚拟上下文与分层记忆管理的操作系统启发	自动“无限精确记忆”或等价的操作系统虚拟内存
CoALA（2023）[2]	认知功能、内部记忆操作与外部行动的框架	所有Agent都采用相同具体架构
Letta Memory Blocks[3]	持续可见的核心记忆块与外部存储组织思路	记忆块自动等于核实事实
LangGraph Persistence[4]	线程级检查点与跨线程Store的职责划分	检查点可以替代长期知识和权限表
Mem0（2025）[5]	信息提取、整合、检索的长期记忆流水线	所有场景都应照搬其方法或数字
Zep / Graphiti（2025）[6]	动态事件、实体关系与时间化检索	图在所有任务上必然优于简单检索

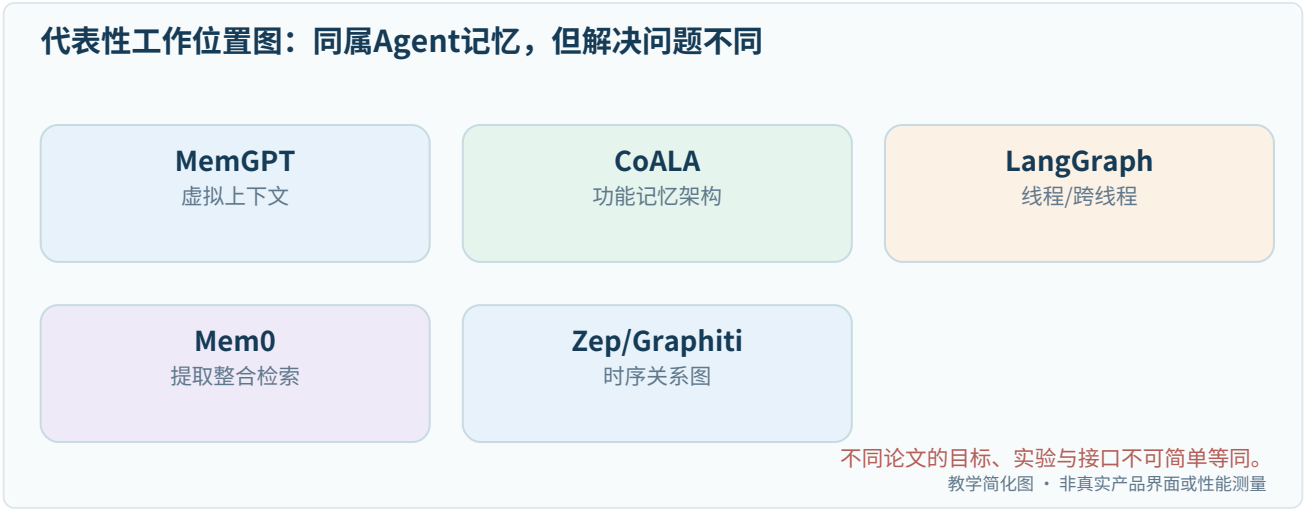


图15 MemGPT、CoALA、LangGraph、Mem0和Zep的作用侧重｜原创教学机制示意

11.1 “混合架构”有多种含义

有时混合指**关键词与向量联合检索**；有时指**文本、图、结构化状态并存**；有时指**短期工作区+长期存储+归档**。这三个都是混合，但混合的层面不一样。科研论文声称“Hybrid Memory”时，首先应辨认具体混合了哪些维度，哪些控制变量也变了。

11.2 论文方法与我们的教学架构关系

MemGPT论文确实提出了操作系统启发的虚拟上下文管理，LangGraph官方确实区分checkpointer与store。本文给出的“规范事件→当前状态→派生多索引→控制器权限核验”是一套**为理解可靠Agent而组合的教学参考架构**，不应归因于某一篇论文的原样实现。

十二、12个常见误区及准确说法

1. **“分层就是按RAM/硬盘速度分层。”** 不全面；Agent还要按语义、状态权威性、跨任务作用域和风险分层。
2. **“工作记忆、情景记忆、语义记忆各要一个数据库。”** 错误；功能类型和物理实现正交。
3. **“长期记忆都应该随时装在上下文里。”** 错误；按需装载与小规模固定可见记忆都有适用场景。
4. **“多路检索投票就能决定谁说得对。”** 错误；重复来源、历史版本及权限差异不能靠简单票数解决。
5. **“图数据库保存了关系，就不需要原文。”** 错误；误抽取必须能回源核验。
6. **“只要原始表更新，向量索引就一定即时一致。”** 错误；独立派生服务可能有更新延迟。
7. **“系统恢复了checkpointer，所有长期知识就自动恢复。”** 错误；检查点和长期Store保存不同状态。
8. **“AI说已获批准，控制器就可以执行。”** 错误；授权以真实权威状态与当前有效范围为准。
9. **“RAG能回答问题，所以等于完整Agent记忆。”** 错误；RAG不自动管理写入、演化、冲突和删除。
10. **“索引越多，召回率和正确率必然越高。”** 错误；新增索引可能引入噪声、维护问题与额外成本。
11. **“备份就是一种在线可检索记忆。”** 错误；备份目标首先是可恢复性。
12. **“混合架构比简单架构复杂，所以论文更有创新。”** 错误；科研价值取决于可证伪的机制、可信比较和实质增量。

十三、理论思考题（只动脑，不写代码）

问题1：为什么工作记忆、情景记忆、向量索引不能画成三个平级、互不关联的数据库，并据此推断谁负责真相？

问题2：第D03天AI建议付费，第D20天研究者禁止付费。D30用向量检索首先找到D03。这是哪个环节可能出错？怎样让系统即使召回D03也不错误执行？

问题3：线程检查点与长期Store有什么区别？分别举出一个需要检查点、一个需要Store的例子。

问题4：把原始事件、当前状态、向量索引、图关系都存进同一关系数据库，还能算“混合架构”吗？为什么？

问题5：一次写入D26后，规范库是v12，索引还是v10。当前请求查询“是否允许新收费操作”，应该读哪个版本？为什么？

问题6：如何理解MemGPT的“内存页换入/换出”比喻？其中最重要的非等价之处是什么？

问题7：课堂预算给出的6项Token分别为800、500、4×180、2×250、250、1200，计算总预算、输入部分和输出预留。如果总窗口是12,000，剩余多少？

问题8：为什么向量和关键词双路检索结果相同，也不能证明证据独立？

问题9：简化串行延迟为40+70+35+600ms，增加120ms图查询后是多少？什么情况下值得加图查询？

问题10：如果系统只处理几十条固定约束，何时不应该引入图数据库？请给出一个优先的简单方案。

问题11：如果一条用户信息要求删除，不再把它加入上下文，与真正完成跨索引/缓存/备份的删除是否相同？为什么？

问题12：假设某个混合方案在知识问答上的Recall@5提高了，但最终工具行动正确率下降，至少提出四种可能的解释。能否只根据Recall@5宣布升级成功？

十四、参考答案与逐题推演

答1：三个词属于不同分类轴。 工作/情景是功能类型；向量是表示与检索方式。一个情景经历可以同时有结构化事件与语义向量；当前工作记忆则是被挑选进入上下文的信息。系统应区分权威原始记录与派生索引，而非让每一层互相覆盖事实。

答2：未必是ANN算法错误。 D03与“付费实验”高度相关，因此候选召回本身可能合理；错误可能发生在过滤/状态核验和推理、执行层。应让召回D03后回源得到“AI建议，未批准”，再查询D20有效约束；实际工具执行时独立检查当前权限状态。不能要求相似度引擎自行理解“授权”。

答3：检查点负责中断恢复中的任务状态，Store负责跨任务可复用内容。 例如论文精读做到第4/6篇，断线后应恢复到尚未完成的第5篇——属于线程检查点；另一个新对话需要记住研究者偏好“默认优先离线核验”——属于跨线程长期Store。一个应用可以将两者底层保存在同一数据库，但其访问作用域不同。[4]

答4：可以。 “混合”既可以是逻辑表示混合，也可以是访问路径混合，不要求部署四套物理服务。例如PostgreSQL既有行数据、全文索引、关系查询，也可通过扩展支持向量索引；其核心仍应是明确的事实主记录和多视图同步规则。

答5：关键授权判断应读取经过核验的规范当前状态v12。 v10可用于粗召回或提示“可能有旧材料”，但绝不能凭旧索引授予权限。可通过version校验和原文回源发现不一致；必要时阻断动作，直到权威状态读取成功。不是所有浏览查询都需强一致，但高风险动作需要明确的新鲜度要求。

答6：有用类比是有限的快速可见区+大的持久储存+装入/换出管理。 但真实虚拟内存能够按确定的地址读取字节，而LLM检索、摘要、再生成是概率性的语义处理，可能误选、遗漏和改写原文。这是不可忽略的非等价之处。[1]

答7：先算4×180=720、2×250=500。 总预算 800+500+720+500+250+1200=3970。输入部分不含预留输出，即 3970−1200=2770。12,000窗口假设下还剩 12000−3970=8030。这8,030不是“可无限追加历史”的许可证，因为额外系统消息、工具数据和生成开销也需要计入。

答8：两条检索路径可能指向同一条原始事件。 同一原文可以同时被关键词倒排索引和向量索引收录，所以两个候选一致并不等于两个独立目击者，更不能把一个上游错误当作“两票赞成”。使用event_id/source_id追溯共同祖先，才知道证据的独立性。

答9：不加图是 40+70+35+600=745ms；加120ms是 865ms。 只有当图查询能够提供简单方案难以获取的跨事件路径信息，且这种质量收益或错误率改善对目标任务有足够价值时，新增120ms与维护成本才可能合理。课堂数字未测量真实框架性能。

答10：先使用带项目ID、时间、来源及状态字段的结构化表，再加必要的关键词/精确索引。 当存在高频复杂关系遍历、源事件和结论间的多跳证据需求，且简单查询实现确有代价或质量缺口，再考虑图视图或图数据库。仅因为图“看起来高级”不是理由。

答11：不同。 “不装载上下文”属于可见性控制；真正删除涉及主记录、派生索引、缓存、事件日志、备份的保留策略和复制副本。删除是否可以/必须覆盖不可变审计和备份，取决于用户要求、法律政策与系统设计；不能宣称将一条记忆标记为隐藏就完成物理清除。

答12：至少四种解释。 ①召回更多旧/无效事实，噪声增加；②重排或上下文拼装截掉关键约束；③模型理解了正确证据却在行动建议中推理错误；④控制器没有检查权限/实时状态；⑤图或索引更新滞后；⑥额外延迟导致任务超时。因此Recall@5只说明候选检索的一方面，不足以证明系统级成功或安全改进。需要分层评估端到端行为及成本。

十五、学完后应该在头脑中留下什么模型？

请尝试不看讲义，在纸上画出这样的路径：**用户与工具产生事件→原始日志保存→提取事实与当前状态→建立全文/向量/关系访问视图→按问题选择检索方式→回源和时间核验→装入有限上下文→模型生成建议→控制器检查并执行→新事件写回。** 若能逐个解释每条箭头为什么存在、出了问题会有什么症状，就已从零散名词记忆进入可以自行推演的架构理解。



图16 从过去事件到未来决策的完整循环｜原创教学机制示意

下一阶段将把重点从“静态的存储与检索架构”推向**记忆的动态更新、冲突、遗忘及自主维护**。届时我们会重新审视：一层记忆的好坏并不取决于它装了多少数据，而取决于它在正确时间给正确任务提供了多少经过核验的有效信息。

参考文献与延伸阅读（截至2026-10-18核对）

[1] Packer, C. et al. (2023/2024). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. 论文提出虚拟上下文管理及分层存储启发。https://arxiv.org/abs/2310.08560

[2] Sumers, T. R. et al. (2023). Cognitive Architectures for Language Agents (CoALA). arXiv:2309.02427. 按记忆功能和行动组织Agent认知架构。https://arxiv.org/abs/2309.02427

[3] Letta Documentation. Memory Blocks (Core Memory). 官方概念文档，解释常驻上下文的可编辑记忆块。https://docs.letta.com/guides/agents/memory

[4] LangGraph Documentation. Persistence / Checkpointers and Stores. 官方说明线程级状态检查点和跨线程Store的区别。https://langchain-ai.github.io/langgraph/concepts/durable_execution/

[5] Chhikara, P. et al. (2025). Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413. 讨论提取、整合、检索和记忆评测。https://arxiv.org/abs/2504.19413

[6] Rasmussen, P. et al. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956. 讨论动态知识、时间化关系与图记忆。https://arxiv.org/abs/2501.13956

[7] LangChain Blog (2024). Launching Long-Term Memory Support in LangGraph. 长期记忆Store的工程设计与简约哲学。https://www.langchain.com/blog/launching-long-term-memory-support-in-langgraph

[8] LangGraph official docs source. Stores. 跨线程持久化信息和索引能力。https://github.com/langchain-ai/docs/blob/main/src/oss/langgraph/stores.mdx

文献阅读提示： 文章摘要与官网文档可以证明作者提出了什么设计，并不自动证明这些方法在所有任务上最优。涉及具体性能对比时需要回到对应论文的模型、任务、基线、指标、代码版本及实验条件；本讲没有宣称已经对各系统做过公平对照实验。